hashlock.

Security Audit

Pruv Finance 4th (Tokenization Platform)

January 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Pruv Finance team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

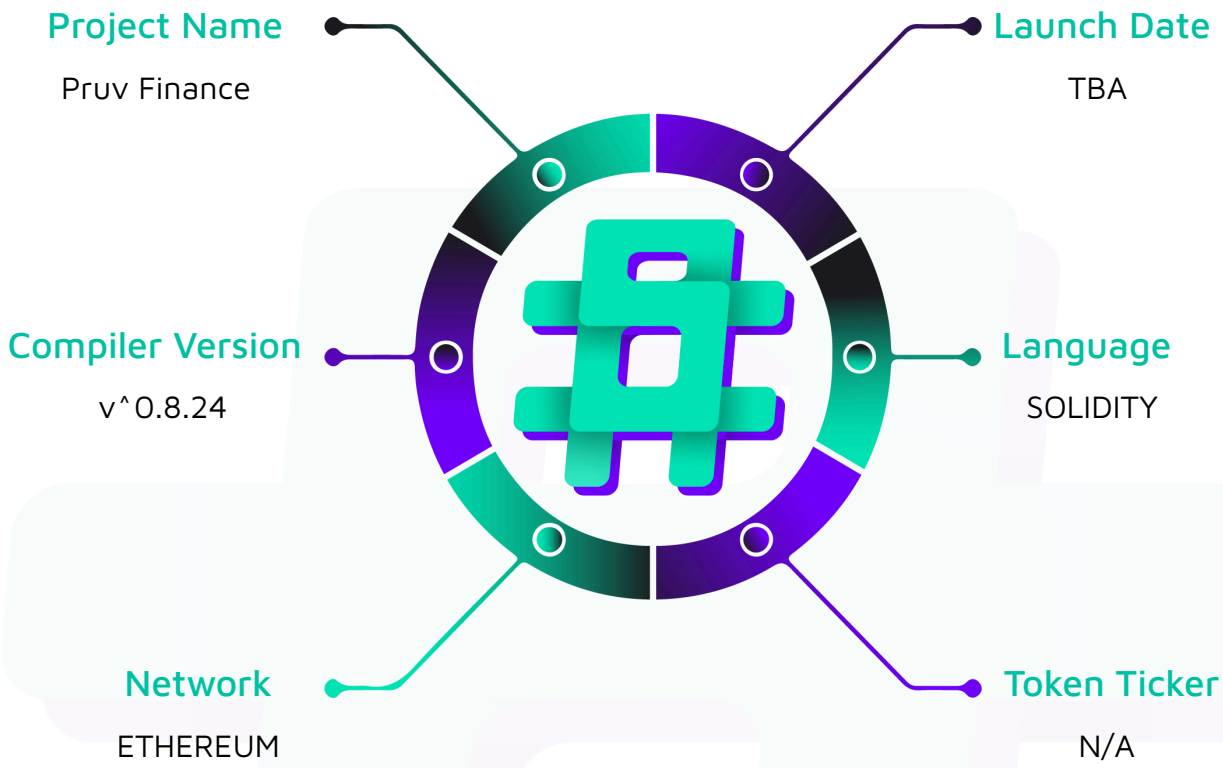
Pruv Finance is an RWA infrastructure layer bridging real-world assets (RWAs) and DeFi. It enables users to access trusted, tokenized assets that are freely tradable and fully composable across decentralized ecosystems.

Pruv streamlines the full lifecycle of real-world assets—from tokenization to distribution and DeFi integration—within a secure, compliant framework. The platform empowers issuers to tokenize assets such as real estate, commodities, and private equity, launch tokenized offerings, and manage them end-to-end.

For investors, Pruv provides seamless access to verified real-world assets that can be subscribed to, held, and redeemed through a transparent, regulated infrastructure. With built-in KYC/AML processes, encrypted transaction records, on-chain token verification, and robust custody solutions, Pruv ensures both compliance and trust at every step of the journey.

Project Name: Pruv Finance
Project Type: Tokenization Platform
Compiler Version: ^0.8.24
Website: https://pruv.finance/
Logo:

Visualised Context:



Project Visuals:

Audit Scope

We at Hashlock audited the solidity code within the Pruv Finance project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Pruv Finance Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**January, 2026**
Contract 1	BridgeStorage.sol
Contract 2	BridgeSystemOperations.sol
Contract 3	BridgeUserOperations.sol
Contract 4	ERC1967Proxy.sol
Contract 5	TokenBridge.sol
Contract 6	TransactionIdTracker.sol
Audited GitHub Commit Hash	0f706821d234515d9b1285cb137ff5733f9958ee
Fix Review GitHub Commit Hash	4577cf335f07438d3b3c01302c81b8bbcd 9686cd

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

1 Medium severity vulnerabilities

7 Low severity vulnerabilities

9 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BridgeStorage.sol Base contract that holds shared data and utilities	**Contract achieves this functionality.**
BridgeSystemOperations.sol Handles operations that only the system wallet can perform: - Release: Sends previously locked tokens to users on the destination chain - Mint: Creates new tokens for users on the destination chain - Both can refund fees if needed	**Contract achieves this functionality.**
BridgeUserOperations.sol Handles operations that regular users can perform: - Lock: User locks tokens and pays a fee; tokens are held in the bridge - Burn: User burns tokens to bridge them back to another chain	**Contract achieves this functionality.**
ERC1967Proxy.sol A proxy contract that forwards all calls to the actual bridge implementation. Enables upgrades without changing the contract address.	**Contract achieves this functionality.**
TransactionIdTracker.sol Prevents replay attacks by tracking which transaction IDs have been used	**Contract achieves this functionality.**
TokenBridge.sol Main contract that coordinates everything: - Single entry point (executeBridgeOperation)	**Contract achieves this functionality.**

routes to the right operation - Role based access: Owner, Admin, System Wallet, Upgrader - Admin can set fees, withdraw treasury funds, pause the bridge, and manage roles - Supports upgrades via UUPS - Ownership transfer uses a two-step process (propose, then accept)	

Code Quality

This audit scope involves the smart contracts of the Pruv Finance project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Pruv Finance project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01]TokenBridge#executeBridgeOperation, TransactionIdTracker#revokeTransactionId - Complete bridge DoS via transaction ID manipulation

Description

The transaction ID system lacks validation and ownership checks, allowing anyone to mark any transaction ID as used. This enables frontrunning and DoS attacks across all bridge operations.

Vulnerability Details

The `transactionIds` mapping keeps track of used transaction Ids. The public user functions like bridge operation `LOCK_WITH_FEE`, bridge operation `BURN`, and `revokeTransactionId()` allows anyone to provide the transaction ID as an input parameter.

The transaction ID is not validated, there's no binding between the transaction ID and the operation metadata or user calling the function. This allows anyone to mark any transaction ID as used enabling frontrunning and DoS attacks across all bridge operations.

User DoS attack:

1. User A prepares a `LOCK_WITH_FEE` operation with `transactionId = "tx-abc-123"` and broadcasts it to the mempool.
2. Attacker monitors the mempool, sees the transaction ID, and calls `revokeTransactionId("tx-abc-123")` or frontruns with the same `transactionId` in their own `LOCK_WITH_FEE` call before User A's transaction executes.
3. User A's transaction reverts with `TransactionIdAlreadyUsed` when it attempts to consume the transaction ID.

4. User A can retry with a different transaction ID, but the attack can be repeated indefinitely.

System Wallet DoS attack:

1. User A executes `LOCK_WITH_FEE` on Chain A with `transactionId = "tx-abc-123"`. User A's tokens are now locked in the bridge contract on Chain A.
2. Attacker monitors Chain A events/logs and sees User A's successful lock operation with `transactionId = "tx-abc-123"`, or monitors Chain B mempool for the system wallet's mint/release operation.
3. Attacker submits a `BURN` transaction or `revokeTransactionId()` to execute first on Chain B, using the same `transactionId = "tx-abc-123"`.
4. When the system wallet attempts to execute `MINT` or `RELEASE` on Chain B, the `_useTransactionId()` call in `_mintTokens()` or `_releaseTokens()` reverts with `TransactionIdAlreadyUsed` because the transaction ID was already consumed by the attacker.

Impact

An attacker can cause permanent denial of service by consuming transactionIds that legitimate users and system wallets need for their bridge operations.

Recommendation

Instead of allowing users to provide the `transactionId` by parameter, consider calculating it deterministically on-chain based on the important parameters of the operation metadata and a salt that could be a random off-chain input or on-chain incremental nonce to ensure uniqueness. E.g. `TXID = hash(operationData, salt)` could be used to generate the transaction ID.

This will require updating the `executeBridgeOperation()` to remove the `transactionId` parameter and calculate it on-chain instead in each user bridge operation.

For `revokeTransactionId()`, entirely remove the ability for it to be publicly callable, similar to the rust implementation in `lib.rs` of `use_transaction_id()`. In case it must

remain public callable, consider adding a validation to check if the caller has the right to revoke the transaction ID. For this to be possible, the transaction ID must be scoped to the user and will require the following changes:

```
- mapping(string transactionId => bool used) public transactionIds;
+ // User --> TXID --> Used?
+ mapping(address => mapping(string => bool)) public transactionIds;
-    function _useTransactionId(string memory transactionId) internal {
-        if (transactionIds[transactionId]) revert TransactionIdAlreadyUsed();
-        transactionIds[transactionId] = true;
-    }
+    function _useTransactionId(address user, string memory transactionId) internal {
+        if (transactionIds[user][transactionId]) revert TransactionIdAlreadyUsed();
+        transactionIds[user][transactionId] = true;
+    }
    function revokeTransactionId(string memory transactionId) external {
-        _useTransactionId(transactionId);
+        _useTransactionId(msg.sender, transactionId);
        emit TransactionIdRevoked(msg.sender, transactionId, block.timestamp);
    }


// Bridge operations will also need to change `_useTransactionId()` calls
// System Operations:
-        _useTransactionId(transactionId);
+        _useTransactionId(recipient, transactionId)
// User Operations:
-        _useTransactionId(transactionId);
```

```
+          // and/or destinationAddress, depending on which address(s) you want to have
dominion

+        _useTransactionId(msg.sender, transactionId);
```

Status

Resolved

Medium

[M-01] TokenBridge#executeBridgeOperation - Users can Gas Grief Bridge System Relayer

Description

There is no limit to the length of the strings in bridge system operations, allowing users to request token mints that would cost the relayer near the block gas limit.

Vulnerability Details

A user makes a dust deposit into the Stellar bridge contract with a very long `email` string of 65k characters on a chain where the gas is very cheap. For example, it may only cost ~0.01 XLM to execute this call from the user side.

The EVM admin sees the Operation event and ends up calling `_mintTokens()` with the extremely long email string. On the EVM, 65k characters which only emit an event costs 21M gas, nearly the gas block limit.

If the off-chain relayer is not sufficiently guarded, users can indefinitely spam bridge requests to drain the relayer's gas funds.

Impact

Allows any user to maliciously drain the off-chain relayers gas funds.

Recommendation

Create a hard limit for all string lengths on the EVM side. (E.g. 100 character maximum).

Also consider adding a dynamic fee or mechanism which always covers the gas costs for the System Wallet.

Status

Resolved

Low

[L-01] Incompatible Tokens are Temporarily Frozen in the Bridge Until the Admin Rescues

Description

Users can lock tokens incompatible with burnable/mintable OpenZeppelin interfaces, disallowing the minting/burning to occur, forcing the admin to manually release tokens back to the user.

Vulnerability Details

Imagine that some popular token like wrapped USDS (Stably's Stellar based USD stablecoin) doesn't have a mint function because it doesn't inherit from `IERC20Mintable`.

The user wants to bridge USDS from Stellar to EVM. They end up calling `lib._lock_tokens()`, locking their USDS.

The EVM admin sees the event and then ends up trying to call `BridgeSystemOperations._mintTokens()`. Because wrapped USDS doesn't use `IERC20Mintable()`, the minting operation fails.

Any token deposited on Stellar which does not adhere to the `IERC20Mintable` interface is not mintable on the EVM side. This also occurs for burn operations.

Impact

Temporary freezing of user funds until admin manually releases incompatible tokens.

Recommendation

Enforce all tokens are mintable/burnable before allowing the user to deposit on the source side of the bridge. This can most easily be done through an on-chain admin-gated whitelist of compatible tokens.

Status

Acknowledged

[L-02] Bridge Fees can Exceed User's Deposit Amount

Description

Flat fees charged during bridge operations cause unusually high fees for small users.

Vulnerability Details

User bridge operations charge a fee during `_burnTokens()` and `LockTokensWithFee()`.

The `feeAmount` is a flat value set by the admin. There is no minimum deposit amount.

Therefore, if the `feeAmount` is $10 and the deposit amount is $5, the user will be paying a 200% fee for a $5 deposit.

Impact

Users may pay 25%-100%+ fees on bridge operations depending on the flat fee.

Recommendation

Consider charging a dynamic fee, fee tiers, and/or include an optional maximum cap.

Status

Acknowledged

[L-03] Users Can Bridge Dust Amounts

Description

Users can deposit 1 wei or other small amounts into the bridge.

Vulnerability Details

There is no check on-chain for minimum bridging amounts. Users can deposit 1 wei or other small amounts into the bridge.

Impact

Users can spam deposits stressing the off-chain admin system. Despite no immediately hard exploit found, dust deposit amounts are unanimously better off being capped at some reasonable minimum.

Recommendation

Consider an admin-set minimum deposit amount per whitelisted token.

Status

Acknowledged

[L-04] BridgeUserOperations - Missing destination chain whitelist validation

Description

The bridge contract does not validate that destination chain identifiers are supported by the bridge, allowing users to send assets to unsupported chains and lock tokens until admin intervention.

Vulnerability Details

The `_validateChainId()` function in `BridgeStorage` only validates the CAIP-2 format of chain identifiers but does not verify whether the chain is actually supported by the bridge.

When a user calls `_lockTokensWithFee()` or `_burnTokens()` with an unsupported destination chain identifier, the operation succeeds as long as the chain identifier format is valid. The tokens are locked in the contract, but since the destination chain is not supported, the system wallet cannot complete the bridge operation on the destination chain. This results in tokens being locked indefinitely until admin intervention.

Impact

Users can accidentally send tokens to unsupported chains, locking funds in the bridge contract until admin intervention

Recommendation

Implement a whitelist mechanism for supported chain identifiers.

Status

Acknowledged

[L-05] TokenBridge#pause - Lack of chain specific pause granularity

Description

The bridge contract implements a pause mechanism that pauses all bridge operations, without the ability to pause specific source or destination chain paths. This design limitation forces administrators to pause all bridge activity when an issue occurs on a single chain.

Vulnerability Details

When `pause()` is called, it sets a global paused flag that affects all operations protected by the `whenNotPaused` modifier which includes all user and system operations.

When an issue occurs on a specific chain (e.g., a vulnerability discovered in the bridge instance on Chain A, Chain A experiencing downtime, or a security incident requiring immediate isolation), administrators must pause all bridge operations across all chains, including healthy ones. This creates unnecessary service disruption and reduces operational flexibility.

Impact

An issue affecting a single bridge chain instance forces administrators to pause all bridge activity across all chains, causing unnecessary service disruption for healthy chains and reducing operational flexibility.

Recommendation

Implement a granular pause mechanism that allows pausing specific chain paths. This could be achieved by implementing separate pause flags for inbound operations (from specific source chains) and outbound operations (to specific destination chains)

Status

Acknowledged

[L-06] BridgeUserOperations - Single fee structure for multiple destination chains

Description

The bridge contract uses a single fee structure for all destination chains, regardless of the actual operational costs that may differ significantly between chains.

Vulnerability Details

The fee parameters apply universally to all bridge operations, without any per destination chain customization.

Different destination chains may have significantly different operational costs for the bridge system. For example, executing transactions on Ethereum mainnet typically costs much more in gas fees compared to lower cost chains like Pruv. This means users bridging to cheaper chains may be overpaying fees, while users bridging to more expensive chains may be underpaying, potentially causing the bridge operator to subsidize those operations.

Impact

Users may overpay or underpay fees depending on the destination chain, as the fee structure does not account for varying operational costs across different chains.

Recommendation

Implement an optional per destination chain fee structure with a default fallback that applies to all chains.

Status

Acknowledged

[L-07] TransactionIdTracker, BridgeStorage - Storage layout inheritance risk in upgradeable contracts

Description

The `transactionIds` mapping is currently defined in `TransactionIdTracker`, which is then inherited by `BridgeStorage`. This creates a storage layout risk where future additions to either contract could cause storage slot collisions in the upgradeable contract system.

Vulnerability Details

`BridgeStorage` inherits from `TransactionIdTracker`. The `transactionIds` mapping is defined in `TransactionIdTracker`, while other state variables like `feeToken`, `feeAmount`, and `lockedBalances` are defined in `BridgeStorage`. This splits storage variables across the inheritance chain.

In upgradeable contracts, storage layout is critical. The order of state variables across the inheritance chain determines their storage slots. When storage variables are split across multiple contracts in the inheritance hierarchy, future modifications become risky. If `TransactionIdTracker` adds new state variables in the future, they will occupy storage slots after `transactionIds` which will collide with the storage of `BridgeStorage`.

Impact

Future contract upgrades could break due to storage layout incompatibilities.

Recommendation

Move the `transactionIds` mapping to `BridgeStorage` and restructure the inheritance hierarchy:

1. Add `transactionIds` mapping to `BridgeStorage` in the state variables section
2. Remove `transactionIds` from `TransactionIdTracker`
3. Update `TransactionIdTracker` to inherit from `BridgeStorage` instead of being inherited by it
4. Update `BridgeUserOperations` and `BridgeSystemOperations` to inherit from `TransactionIdTracker` instead of `BridgeStorage`

This ensures all storage variables are centralized in `BridgeStorage`, making the storage layout predictable and upgrade-safe.

Status

Resolved

QA

[Q-01] TokenBridge#withdrawTreasury - Redundant Checks

Description

There are redundant and nonsensical checks in `withdrawTreasury()`.

Vulnerability Details

The `withdrawTreasury()` function is meant to withdraw all accumulated fees and any excess donation/miscellaneous funds in excess of the actual user locked funds.

We never want the admin to have the ability to accidentally clear out all $7 of accumulated fees where the `amount` (difference between total balance and locked user funds) is only $5.

This would send the admin $5 but remove $7 of theoretically claimable accumulated fees.

The good news is that the state of `amount < fee` is provably unreachable. It's impossible for fees to exceed `balance - locked` by definition. The fees only ever accumulate when the contract's balance increases by the same amount or more. And the same for refunds, the fees reduce at the same rate that the contract's balance decreases.

Additionally, the locked balance always strictly increases/decreases the contract's balance by the exact locked amount deposited/released.

We can understand `balance - locked` as:

"All current fee token donations + all current fees - current locked balance".

Therefore, it's impossible for `fee` to exceed the `amount` to be withdrawn by the treasury.

There is a case where a user directly donates fee tokens to the balance, such that `balance` = 1200, `locked` = 1000, `fees` = 50. This scenario works as intended, making `amount` = 200, sending the entire excess of 200 to the admin while correctly zeroing the accumulated fees mapping back to 0.

Impact

Confusing/Redundant checks and small waste of gas.

Recommendation

We can remove the latter check entirely, which may have originally meant to be `||` instead of `&&`.

```
- if (amount == 0 && amount < fee) revert InvalidAmount();

+ if (amount == 0) revert InvalidAmount()
```

This line may also be deleted:

```
- if (balance <= locked) revert AmountUnderflow();
```

As it's impossible for the balance to be less than the `locked`, as the `locked` is always increased/decreased the same as the contract's balance, assuming no FoT tokens. The `amount == 0` check explicitly reverts already when `balance == locked`.

Status

Resolved

[Q-02] BridgeStorage#_parseAddress - Consider Stricter Address Parsing

Description

It's possible for invalid addresses to silently parse into valid EVM addresses.

Vulnerability Details

`BridgeStorage._parseAddress()` will silently parse a 42 character string that does not start with 0x, and still return the last 40 characters.

E.g. `0xd9733ccff94817a7ac45758cbd3c312350e89b39` and `ZZd9733ccff94817a7ac45758cbd3c312350e89b39` are different 42 character strings that both parse to `d9733ccff94817a7ac45758cbd3c312350e89b39`.

Impact

Invalid addresses can be silently parsed into valid addresses.

Recommendation

Ensure if the length of the input is 42 characters, the first two characters are exactly `0` and `x`.

```
if (stringBytes.length == 42 && (stringBytes[0] != '0' || stringBytes[1] != 'x')) {

    revert InvalidHexPrefix();
```

Status

Resolved

[Q-03] BridgeStorage#_validateChainId - Consider Stricter Chain Id Validation

Description

Chain ID validation only checks for the existence of at least one colon in the string.

Vulnerability Details

CAIP-2 requires exactly 1 colon, with a maximum of 41 characters.

The following are examples of invalid ChainId's that would currently pass on-chain validation:

- ":::"
- "1:"
- ":"
- "lolololololololololololololololololololololololololololololololololol:ol"
- "q:1:2:55:jjj4:5:::"""//::6:KMx7???#,.,.,.,.##(())()?"

Impact

Invalid Chain IDs will validate, which may confuse off-chain components.

Recommendation

```
function _validateChainId(string memory chainId) internal pure {

    bytes memory b = bytes(chainId);

    if (b.length < 3 || b.length > 41) revert InvalidChainIdentifier();

    // Find colon position

    uint colonPos = 0;
```

```
    uint colonCount = 0;

    for (uint i = 0; i < b.length; i++) {

        if (b[i] == ':') {

            colonPos = i;

            colonCount++;

        }

    }

    // Exactly one colon required

    if (colonCount != 1) revert InvalidChainIdentifier();



    // Namespace must be 3-8 chars

    if (colonPos < 3 || colonPos > 8) revert InvalidChainIdentifier();

    // Reference must be 1-32 chars

    uint refLength = b.length - colonPos - 1;

    if (refLength < 1 || refLength > 32) revert InvalidChainIdentifier();

    // You may also validate only valid characters (a-Z, 0-9, :) are used

}
```

Status

Resolved

[Q-04] BridgeUserOperations#_validateChainId - Should use `burnFrom()` To Burn Tokens

Description

The bridge performs two calls to burn tokens, but can be done in one.

Vulnerability Details

During `_burnTokens()`, the function calls `safeTransferFrom()` and `burn()` separately, but can use `burnFrom()` to save gas and reduce code complexity.

Impact

Slightly more complex code.

Recommendation

```
-       IERC20(_token).safeTransferFrom(msg.sender, address(this), amount);
-       IERC20Burnable(_token).burn(amount);
+       IERC20Burnable(_token).burnFrom(msg.sender, amount);
```

Status

Acknowledged

[Q-05] TokenBridge#updateAccumulatedFee - Remove Function Per Comments

Description

updateAccumulatedFee has been marked to be removed. Ensure its removal before deploying the contract.

Vulnerability Details

updateAccumulatedFee has been marked to be removed.

Impact

Automatic updating of accumulated fees by the admin should not persist onto mainnet.

Recommendation

Remove updateAccumulatedFee.

Status

Resolved

[Q-06] Proxy In The Middle - Attack Prevention

Description

There is a novel proxy stealth sandwich attack discovered in 2025, denoted the "proxy hijack" or "CPIMP" (Clandestine Proxy In the Middle of Proxy).

The EVM based deployment and upgrade scripts were checked if the deployment and upgrade processes were vulnerable to this attack. They were safe, but it needs to be noted what should not be changed during deployment.

This attack is not possible for Stellar, as the entire upgrade process is completely different.

Vulnerability Details

In short, a malicious user can frontrun deployment scripts to deploy their own "stealth" proxy between the deployment of the legitimate proxy and the initialization of the implementation contract to the proxy. If there is a time gap between proxy deployment and the linking of the implementation to the proxy, then the attack will go unnoticed, with the attacker controlling the actual proxy state, while off-chain resources like Etherscan show everything as normal.

The CPIMP attack won't work on the current deploy scripts because `const proxy = await BridgeProxy.deploy(implAddress, initData);` atomically deploys proxy and initializes the implementation with the proxy. There is no gap/await time between proxy being deployed and it being initialized with the implementation.

Impact

N/A

Recommendation

Do NOT change the deployment script to ever initialize the proxy with the implementation in a separate transaction.

To check if your protocol has been infected by a CPIMP attack, simply look at the number of `delegatecall`'s from the deployment tx hash. If there is more than a single `delegatecall`, then the project is taken over.

Status

Resolved

[Q-07] Multiple contracts - Floating pragma

Description

Multiple contracts use floating pragma statements pragma solidity ^0.8.28, instead of fixed version pragmas.

Vulnerability Details

Floating pragmas allow contracts to be compiled with different compiler versions than those tested, potentially introducing unidentified security issues.

Impact

Different pragma versions in test and mainnet may pose unidentified security issues.

Recommendation

Consider locking the pragma version to the specific version pragma solidity 0.8.28.

Status

Resolved

[Q-08] BridgeSystemOperations, TokenBridge - Unused imports

Description

The contracts contain unused import statements that are not referenced anywhere in the code.

Vulnerability Details

The contracts `BridgeSystemOperations.sol` and `TokenBridge.sol` import `Strings` and `ERC1967Utils` from OpenZeppelin but never use them.

Impact

Unused imports reduce code clarity without affecting functionality.

Recommendation

Remove the unused import statements from both contracts to improve code cleanliness.

Status

Resolved

[Q-09] BridgeStorage - Unused Errors

Description

The `BridgeStorage` contract declares five custom errors that are never used in the codebase.

Vulnerability Details

The `BridgeStorage` contract defines several custom errors in the errors section that are declared but never referenced in any contract function. These unused errors are:

- UnsupportedToken
- UnsupportedChain
- InvalidReleaseOnSameChain
- InvalidSourceChain
- InvalidRefund

Impact

Unused error declarations reduce code clarity and maintainability without affecting functionality.

Recommendation

Remove the unused error declarations to improve code cleanliness.

Status

Resolved

Centralisation

The Pruv Finance project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Pruv Finance project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au